

Symbol Type - Simple Hinglish Summary

Source: <https://javascript.info/symbol>

Main baat

Specification ke hisaab se object property keys ke roop me sirf **do primitive types** use ho sakte hain:

- **string**
- **symbol**

Koi aur type (jaise number) use karo to wo **automatically string** me convert ho jata hai.

Yaani `obj[1] = obj["1"]` aur `obj[true] = obj["true"]`.

Ab tak humne sirf strings use kiye. Ab **symbols** dekhte hain.

1. Symbols kya hain?

Symbol ek unique identifier ko represent karta hai. Ise `Symbol()` se banate hain:

```
javascript  
  
let id = Symbol();
```

Banate waqt **description (symbol name)** de sakte hain, jo zyadatar **debugging** ke kaam aati hai:

```
javascript  
  
// id ek symbol hai jiska description "id" hai  
let id = Symbol("id");
```

Symbols hamesha unique hote hain. Bilkul same description ke kai symbols banao to bhi wo **alag values** hote hain. Description sirf ek label hai jo kisi cheez ko affect nahi karti.

```
javascript  
  
let id1 = Symbol("id");  
let id2 = Symbol("id");  
  
alert(id1 == id2); // false
```

(Ruby jaisi languages ke symbols se confuse mat hona. JavaScript ke symbols alag hain.)

Short me: symbol ek **"primitive unique value"** hai jiska optional description hota hai.

Symbols string me auto-convert nahi hote

JS me zyadatar values string me implicitly convert ho jati hain (jaise `alert` lagbhag kuch bhi dikha deta hai). **Symbols special hain, wo auto-convert nahi hote.**

```
javascript

let id = Symbol("id");
alert(id); // TypeError: Cannot convert a Symbol value to a string
```

Ye ek **"language guard"** hai, kyunki strings aur symbols bunyadi taur par alag hain aur galti se ek dusre me convert nahi hone chahiye.

Symbol dikhana ho to `.toString()` explicitly call karo:

```
javascript

let id = Symbol("id");
alert(id.toString()); // Symbol(id)
```

Ya sirf description ke liye `symbol.description` property use karo:

```
javascript

let id = Symbol("id");
alert(id.description); // id
```

2. "Hidden" properties

Symbols se object me **"hidden" properties** bana sakte hain jinhe code ka koi aur hissa **galti se access ya overwrite nahi kar sakta**.

Maan lo `user` objects kisi **third-party code** ke hain aur hume unme identifiers add karne hain. Symbol key use karte hain:

```
javascript
```

```
let user = { // dusre code ka hai
  name: "John"
};

let id = Symbol("id");

user[id] = 1;

alert( user[id] ); // symbol ko key bana kar data access kar sakte hain
```

String `"id"` ke bajay `Symbol("id")` ka fayda?

`user` objects dusre codebase ke hain, isliye unme fields add karna **unsafe** hai, kyunki hum us codebase ke pehle se defined behavior ko affect kar sakte hain. Lekin **symbols galti se access nahi ho sakte**. Third-party code ko naye symbols ka pata hi nahi hoga, isliye `user` objects me symbols add karna safe hai.

Ab maan lo dusri script ko bhi `user` me apni identifier chahiye. Wo apna `Symbol("id")` bana sakti hai:

```
javascript

// ...
let id = Symbol("id");

user[id] = "Their id value";
```

Hamari aur unki identifiers me **koi conflict nahi hoga**, kyunki symbols hamesha alag hote hain, chahe naam same ho.

Lekin agar `"id"` string use karte, to conflict hota:

```
javascript

let user = { name: "John" };

// Hamari script "id" property use karti hai
user.id = "Our id value";

// ...Dusri script ko bhi apne liye "id" chahiye...

user.id = "Their id value"
// Boom! dusri script ne overwrite kar diya!
```

Object literal me symbols

Object literal `{...}` me symbol use karna ho to use **square brackets** me likhna padta hai:

```
javascript

let id = Symbol("id");

let user = {
  name: "John",
  [id]: 123 // "id": 123 nahi
};
```

Kyunki hume key ke roop me **variable** `id` ki value chahiye, string `"id"` nahi.

`for...in` symbols ko skip karta hai

Symbolic properties `for...in` loop me شامل nahi hoti.

```
javascript

let id = Symbol("id");
let user = {
  name: "John",
  age: 30,
  [id]: 123
};

for (let key in user) alert(key); // name, age (symbols nahi)

// symbol se seedha access kaam karta hai
alert( "Direct: " + user[id] ); // Direct: 123
```

`Object.keys(user)` bhi inhe ignore karta hai. Ye "symbolic properties ko chhupane" ke general principle ka hissa hai. Dusri script ya library hamare object par loop chalaye to bhi wo anjaane me symbolic property access nahi karegi.

Lekin `Object.assign` dono (string aur symbol) properties copy karta hai:

```
javascript
```

```
let id = Symbol("id");
let user = {
  [id]: 123
};

let clone = Object.assign({}, user);

alert( clone[id] ); // 123
```

Isme koi contradiction nahi, ye **by design** hai. Jab hum object clone ya merge karte hain, aam taur par hume **saari properties** (symbols jaise `id` bhi) copy karni hoti hain.

3. Global symbols

Aam taur par saare symbols alag hote hain, chahe naam same ho. Lekin kabhi kabhi hume chahiye ki **same naam wale symbols same entity** ho. Jaise application ke alag hisse symbol `"id"` se bilkul ek hi property access karna chahein.

Iske liye **global symbol registry** hai. Usme symbols bana aur baad me access kar sakte hain, aur ye guarantee deta hai ki **same naam se baar baar access karne par bilkul wahi symbol milega**.

Registry se symbol lene (na ho to banane) ke liye `Symbol.for(key)`:

Ye call global registry check karti hai, agar `key` naam ka symbol hai to wahi return karti hai, warna naya `Symbol(key)` banakar registry me us `key` se store karti hai.

```
javascript

// global registry se lo
let id = Symbol.for("id"); // symbol na ho to ban jayega

// dobara lo (shayad code ke kisi aur hisse se)
let idAgain = Symbol.for("id");

// wahi symbol
alert( id === idAgain ); // true
```

Registry ke symbols ko **global symbols** kehte hain. Application-wide symbol chahiye jo code me har jagah accessible ho, to ye wahi hain.

(Ruby jaisi languages me ek naam ka ek hi symbol hota hai. JS me ye sirf global symbols ke liye sach hai.)

Symbol.keyFor

Symbol.for(key) naam se symbol deta hai. Iska ulta (symbol se naam) Symbol.keyFor(sym) deta hai:

```
javascript

// naam se symbol lo
let sym = Symbol.for("name");
let sym2 = Symbol.for("id");

// symbol se naam lo
alert( Symbol.keyFor(sym) ); // name
alert( Symbol.keyFor(sym2) ); // id
```

Symbol.keyFor andar se global registry use karta hai, isliye ye **non-global symbols ke liye kaam nahi karta**, unke liye undefined milta hai.

Lekin **saare symbols ki description property** hoti hai:

```
javascript

let globalSymbol = Symbol.for("name");
let localSymbol = Symbol("name");

alert( Symbol.keyFor(globalSymbol) ); // name, global symbol
alert( Symbol.keyFor(localSymbol) ); // undefined, global nahi

alert( localSymbol.description ); // name
```

4. System symbols

JavaScript ke andar kai **"system" symbols** use hote hain, jinhe hum apne objects ke alag alag pehlu fine-tune karne ke liye use kar sakte hain.

Ye specification ki "Well-known symbols" table me hain:

- Symbol.hasInstance
- Symbol.isConcatSpreadable
- Symbol.iterator
- Symbol.toPrimitive
- ...aur bhi

Jaise Symbol.toPrimitive se object se primitive me conversion describe kar sakte hain. Baaki symbols bhi aage jab related features seekhenge tab familiar ho jayenge.

Summary

`Symbol` unique identifiers ke liye ek primitive type hai.

- `Symbol()` call se bante hain, optional description (naam) ke saath.
- Symbols **hamesha alag values** hote hain, chahe naam same ho. Same naam wale symbols ko barabar rakhna ho to **global registry** use karo: `Symbol.for(key)` (na ho to banata hai). Same `key` ke saath `Symbol.for` ke kai calls bilkul wahi symbol dete hain.

Symbols ke do main use cases:

1. "Hidden" object properties. Jo object kisi aur script ya library ka hai, usme property add karni ho to symbol bana kar use property key banao. Symbolic property `for..in` me nahi dikhti, to galti se baaki properties ke saath process nahi hogi. Aur dusri script ke paas hamara symbol nahi hai, isliye wo seedha access bhi nahi kar sakti. Isse property **galti se use ya overwrite hone se surakshit** rehti hai. Yaani objects me aisi cheezein "chupke se" rakh sakte hain jo dusron ko nahi dikhni chahiye.

2. System symbols. JS ke kai system symbols `Symbol.*` ke roop me milte hain. Inse built-in behaviors badal sakte hain. Jaise aage `Symbol.iterator` (iterables ke liye) aur `Symbol.toPrimitive` (object-to-primitive conversion ke liye) use karenge.

Dhyan: Technically symbols **100% hidden nahi hote**. Built-in method

`Object.getOwnPropertySymbols(obj)` saare symbols de deta hai. Aur `Reflect.ownKeys(obj)` object ki **saari keys** (symbolic bhi) return karta hai. Lekin zyadatar libraries, built-in functions aur syntax constructs in methods ko use nahi karte.

Quick Summary

Baat	Yaad rakho
Symbol banana	<code>let id = Symbol("id");</code>
Uniqueness	<code>Symbol("id") == Symbol("id")</code> → <code>false</code>
String me convert	Auto nahi hota (TypeError). <code>.toString()</code> ya <code>.description</code> use karo
Object me use	<code>user[id] = 1;</code> ya literal me <code>{ [id]: 123 }</code>
<code>for..in</code>	Symbols ko skip karta hai
<code>Object.keys</code>	Symbols ko ignore karta hai
<code>Object.assign</code>	Symbols copy karta hai
Global symbol	<code>Symbol.for("id")</code> (same naam = same symbol)
Naam nikalna	<code>Symbol.keyFor(sym)</code> (sirf global symbols ke liye)
System symbols	<code>Symbol.iterator</code> , <code>Symbol.toPrimitive</code> , etc.
Poori keys	<code>Reflect.ownKeys(obj)</code> (symbols bhi)

Yaad rakho: Symbols ka sabse bada fayda: dusre code ke objects me **conflict ke bina** apni "hidden" property jodna.